PEC1

Estructura de computadores

Programa

2025 s2

Estudios de informática, multimedia y comunicación

EIMT.UOC.EDU

UOC
Universitat Oberta de Catalunya

La presente PEC1 contiene 5 preguntas y representa el 50% de la nota de la evaluación continua.

Como podréis ver, los ejercicios son muy parecidos a los que habéis hecho durante estos días, en los que además habéis podido dar las soluciones, comentarlas y plantear dudas en el foro. Esta PEC es **individual**, **evaluable** y por tanto no puede comentarse.

Competencias

Las competencias específicas que persigue la PEC1 son:

- [13] Capacidad para identificar los elementos de la estructura y los principios de funcionamiento de un ordenador.
- [14] Capacidad para analizar la arquitectura y organización de los sistemas y aplicaciones informáticos en red.
- [15] Conocer las tecnologías de comunicaciones actuales y emergentes y saberlas aplicar convenientemente para diseñar y desarrollar soluciones basadas en sistemas y tecnologías de la información.

Objetivos

Los objetivos de la siguiente PEC son:

- Conocer el juego de instrucciones de la máquina CISCA.
- Conocer los modos de direccionamiento de la máquina CISCA.
- Traducir pequeños programas a ensamblador.
- Comprender la codificación interna de las instrucciones de ensamblador.
- Describir las micro operaciones involucradas en la ejecución de una instrucción de ensamblador.

Enunciado

Responder cada pregunta o apartado en el recuadro correspondiente.

Recursos

Podéis consultar los recursos disponibles en el aula, pero no hacer uso del foro.

Criterios de valoración

La **puntuación** de cada pregunta y los **criterios de evaluación** los encontraréis a cada pregunta.

Formato y fecha de entrega

- La PEC1 podéis entregarla en el apartado de **entrega de actividades** con el nombre **apellido1_apellido2_nombre_PEC1 (pdf).**
- **La fecha límite de entrega es el 16/03/2026.**

Enunciado

Pregunta 1 (2 puntos)

Suponed que el estado inicial del computador (el valor que contienen los registros, posiciones de memoria y bits de resultado justo antes de comenzar la ejecución de cada fragmento de código, de cada apartado) es el siguiente:

R0= 0000BEE0h	M(00000100h)= 00000030h
R1= 00000010h	M(00000110h)= 00000005h
R2= 00000100h	M(0000BEE0h)= 12345678h
R3= 00000000h	M(00000200h)= FFFFFFFFh
R4= 00000000h	M(00000300h)= 0000FF00h

- Bits de resultado del registro de estado: Z=0, S=0, C=0, V=0
- Registros especiales: suponemos que el PC apunta al inicio del fragmento de código de cada apartado.

¿Cuál será el estado del computador después de ejecutar cada uno de los siguientes fragmentos de código? Indicad solamente el contenido (**en hexadecimal**) de los registros y posiciones de memoria que se hayan modificado como resultado de la ejecución del código. Indicad el valor final de todos los bits de resultado. (No os pedimos que indiquéis el valor del PC después de ejecutar el código y por eso no os hemos dado el valor inicial del PC, donde comienza cada fragmento de código).

Suponed que la **dirección** simbólica **A** vale **00000200h**

Importante: dejad claramente indicado (**preferiblemente resaltado o en otro color**) **al final de cada apartado el valor final** de los registros y direcciones de memoria modificadas, así como los bits de estado.

Desarrollad la respuesta, no solo el resultado final. ☐

a)

```
MOV R3, [R2]
ADD R3, [R2+10h]
XOR R3, [A]
```

NOT R3

-MOV R3, [R2]
R2 = 00000100h
R3 =[R2] = [00000100h]= 00000030h
R3 = 00000030h

Las flags permanecen en 0

-ADD R3, [R2+10h]
[R2+10h] = 00000100h+10h = 00000110h
R3 = 00000030h + [00000110h] = 00000030h +00000005h
R3 = 00000035h

Las flags permanecen en 0

-XOR R3, [A]
R3 = 00000035h
A vale **00000200h**
[A]=[**00000200h**]= FFFFFFFFh

Luego 00000035h XOR FFFFFFFFh ->
Bit a bit sería (Voy a comvertir de hex a binario con la calculadora, aunque A es todo unos.
00000035h = 00000000000000000000000000110101
FFFFFFFFh = 11111111111111111111111111111111

11111111111111111111111111001010 b

R3 = FFFFFFCA h

El bit más significativo tiene valor 1, luego S=1. El resto siguen igual.

- NOT R3
11111111111111111111111111001010 b invertido ->
00000000000000000000000000110101 b

R3= 35 h

NOT no cambia los bits de resultado

Solución:	R3=	00000035	h
Z= 0 , S= 1 , C= 0 , V=0			

b)

```
MOV R1, [R0]
SUB R1, [R2]
ADD R1, [A]
NOT R1
```

-MOV R1, [R0]
R1 = [R0] = [0000BEE0h] = 12345678h
R1 = 12345678h

Flags permanecen en 0

-SUB R1, [R2]
[R2] = [00000100h] = 00000030h
R1 = 12345678h
Hacemos la resta. Podríamos hacerla también en Ca2, voy a hacerla directamente.
R1 = 12345678h- 00000030h= 12345648h

Flags siguen en 0

-ADD R1, [A]
[A]=[**00000200h**]= FFFFFFFFh = (-1) en Ca2
R1 = 12345648h

Hacemos la suma
R1 = 12345648h + (-1) = 12345647h

C=1 porque en la suma se ha sobrepasado el bit 31.

-NOT R1
R1 = 12345647h = 00010010001101000101011001000111 b
NOT R1= 11101101110010111010100110111000 b

Guardamos el resultado en R1, luego
R1 = EDCBA9B8 h

NOT no cambia las Flags

Solución: R1 = EDCBA9B8 h

Z= 0 , S= 0 , C= 1, V=0

c)

```
START:  CMP R2, 0000h
        JE  END
        ADD R4, [R2]
        SUB R2, 0100h
        JMP START
END:
```

R0= 0000BEE0h	M(00000100h)= 00000030h
R1= 00000010h	M(00000110h)= 00000005h
R2= 00000100h	M(0000BEE0h)= 12345678h
R3= 00000000h	M(00000200h)= FFFFFFFFh
R4= 00000000h	M(00000300h)= 0000FF00h

```
START:  CMP R2, 0000h
```

R2= 00000100h
Si restamos 00000100h – 0 = 00000100h
No se guarda el resultado

No cambian las flags

```
        JE  END
```

La flag de Z no está activada, luego seguimos

```
        ADD R4, [R2]
```

R4= 00000000h
R2= 00000100h
M(00000100h)= 00000030h
Sumamos 00000000h + 00000030h = 00000030h
R4 = 00000030h

Flags permanecen igual

```
        SUB R2, 0100h
```

R2= 00000100h
Si restamos 00000100h - 0100h = 0h

```
R2 = 0h

Se activa la flag de Z

          JMP START
Saltamos a la etiqueta START

  (2) START:  CMP R2, 0000h
R2= 0h
Comparamos restando 0h - 0h = 0h
No guardamos el resultado

Se activa la flag de 0

  (2) JE  END
La flag de Z está activada, luego saltamos a la etiqueta END.

  END:
Se termina el progama
```

Solución: R2 = 0 h,
R4 = 00000030 h

Z= 1, S= 0, C= 0, V=0

Criterios de valoración. Los apartados a) y b) valen 0,5 puntos cada uno y el c) vale 1 punto. La valoración de cada apartado es del 100% si no hay ningún error en la solución del apartado, es del 50% si el contenido de las posiciones de memoria y registros modificados es correcto, pero hay algún error en el contenido de uno o varios de los bits de resultado, y es del 0% si hay algún error en alguna posición de memoria o registro modificado.

Pregunta 2 (2 puntos)

En el código de alto nivel `M` es una variable de tipo vector de 10 elementos. Cada elemento de la matriz es un entero de 32 bits. En el programa ensamblador la matriz se encuentra almacenada a partir de la dirección simbólica `M`, en posiciones consecutivas (`M[0]`, en la dirección simbólica `M`, el siguiente, `M[1]`, en la dirección `M+4`, etc.).

```
I := 0;
J := 9;
MOVES := 10;

WHILE (I < J)&&(MOVES>7) DO {
    IF (M[I] = 0) {
        T := M[I];
        M[I] := M[J];
        M[J] := T;
        MOVES := MOVES - 1;
        J := J - 1;
    } ELSE {
        I := I + 1;
    }
}
```

Rellena los espacios de la propuesta de programa ensamblador que se muestra a continuación para conseguir el resultado deseado.

Utilizaremos el registro R1 para recorrer el vector hacia adelante (hace la función de "i"), el registro R2 para recorrerlo desde atrás (hace la función de "j") y R5 para el contador de movimientos.

Solución:

```
  MOV R1, 0          ; I
  MOV R2, 36_         ; J, 9 por 4
  MOV R5, 10         ; MOVES

LOOP:
  CMP R1, R2
  JGE_ END
  CMP R5,7
  JLE_ END

  MOV R3, [M + R1]_       ; M[I]
  CMP R3, 0
  JNE ELSE           ; si M[I] != 0, avanzar I

  ; swap M[I] con M[J]
  MOV R4, [M+R2]     ; R4 es T
  MOV [M+R1]_ , R4
  MOV [M+R2], R3__
  DEC_ R5
  SUB R2, 4_  ;Decrementamos 1 posición, que son 4 bytes

  JMP LOOP

ELSE:
  ADD R1,4____  ; Añadimos 1 posición, que son 4 bytes
  JMP LOOP

END:
  MOV [MOVES]_, R5
```

Criterios de valoración.

2 puntos. Se pierden 0,5 puntos por cada instrucción incorrecta.

Pregunta 3 (3 puntos)

Traducid a lenguaje máquina el fragmento de código en lenguaje ensamblador que os proponemos en la tabla inferior. Suponed que la primera instrucción del código se ensambla a partir de la dirección **00070000h** (que es el valor del PC antes de empezar la ejecución del fragmento de código). Suponed que la dirección simbólica Y vale **001234A0h**.

			Bk per k=0..10										
Dirección	**Eti**	**Ensamblador**	0	1	2	3	4	5	6	7	8	9	10
00070000 h		CMP R10,[Y]	26	1A	20	A0	34	12	00				
00070007 h	E1:	JE E2	41	60	0D	00							
0007000B h		SUB R10,[Y+R5]	21	1A	55	A0	34	12	00				
00070012 h		JMP E1	40	00	07	00	07	00					
00070018 h	E2:	MUL R4,[R3]	22	14	33								

JE E2

Sacamos el código de JE E2 de restar el desplazamiento a E2 en Little Endian.

No lo podemos sacar hasta que lleguemos a E2, por lo que reservamos las casillas.

Al volver, **00070018h (E2) - 0007000Bh (PC) = 000D h -> En little endian 0D00**

SUB R10,[Y+R5]

Usamos el código 55 porque es indexado con registro R5 ->55h

MUL R4,[R3]

Usamos el código 55 porque es indirecto a registro R3 ->33h

Criterios de valoración. Cada instrucción ensamblada incorrectamente resta 0.5 puntos. Una instrucción está incorrectamente ensamblada si no se escribe el valor o se escribe un valor incorrecto de uno o varios de los dígitos hexadecimal que codifican la instrucción en lenguaje máquina. Además, se resta 0.5 puntos si hay algún error en la columna que indica las direcciones de memoria en las que comienza cada instrucción.

Pregunta 4 (2 puntos)

El *ciclo de ejecución* de una instrucción se divide en 3 fases principales
1) Lectura de la instrucción
2) Lectura de los operandos fuente
3) Ejecución de la instrucción y almacenamiento del operando destino

Dar la secuencia de micro-operaciones que hay que ejecutar en cada fase para las siguientes instrucciones del código codificado en la pregunta anterior.

			Bk per k=0..10										
Dirección	**Eti**	**Ensamblador**	0	1	2	3	4	5	6	7	8	9	10
00070000h		CMP R10,[Y]	26	1A	20	A0	34	12	00				
00070007h	E1:	JE E2	41	60	0D	00							
0007000Bh		SUB R10,[Y+R5]	21	1A	55	A0	34	12	00				
00070012h		JMP E1	40	00	07	00	07	00					
00070018h	E2:	MUL R4,[R3]	22	14	33								

MUL R4,[R3]

Fase	Micro-operaciones
1	PC = 00070018 h, read -> MAR = 00070018 h Memoria -> MBR = 00331422 h PC = 00070018 h + 3 => PC = 0007001B h MBR = 00331422 h -> IR = 00331422 h
2	

	Direccionamiento indirecto a registro, tenemos que acceder a la memoria. R3, read -> MAR ; R3 en MAR para acceder a memoria Memoria -> MBR ; Rescatamos el dato y lo dejamos en el buffer
3	R4 * MBR -> R4 ; Multiplicamos el dato por el registro y sobreescribimos el registro.

40	00	07	00	07	00

JMP E1

Fase	Micro-operaciones
1	PC = 00070012 h, read -> MAR = 00070012 h Memoria -> MBR = 000700070040 h PC = 00070012 h + 6 => PC = 00070018 h MBR = 000700070040 h -> IR = 000700070040 h
2	No hay que hacer nada en esta fase
3	IR(dirección)= 00070007 h --->PC = 00070007 h

Criterios de valoración. Si todo está correcto se obtienen 2 puntos. Cada instrucción es independiente y vale 1 punto. Se resta 0.5 por cada fallo dentro de la misma instrucción.

Pregunta 5 (1 punto)

Responde a las siguientes preguntas:

Pregunta 1.
¿En qué consiste el "direccionamiento indirecto a registro" y por qué es generalmente más lento que el "direccionamiento directo a registro".

-Direccionamiento Indirecto a registro: La instrucción nos indica un registro, y accedemos a la dirección de memoria a la que apunte el registro donde se encuentra el dato con el que queremos trabajar.

-Direccionamiento Directo a registro: La instrucción nos indica un registro en el operando, y el propio dato está contenido en el registro, por lo que el acceso es inmediato.

El direccionamiento indirecto es más lento que el direccionamiento directo, el cual requiere de menos pasos y se queda todo en el procesador y los registros, por lo que es más rápido el directo.

Pregunta 2.
Qué diferencias principales hay entre las unidades de control "cableadas" y las "microprogramadas" en términos de flexibilidad y velocidad?

Que las cableadas tienen menos flexibilidad, pero son más rápidas, y las microprogramadas al revés.

-Cableadas: Más rápida. Es puro hardware que transforma la entrada directamente en señales de salida.
Es más rígida porque para cambiar la lógica hay que cambiar el propio hardware.

-Microprogramadas. Es más flexible porque tiene un software que le permite cambiar las instrucciones a su antojo sin alterar su hardware.

Pero es más lenta porque las instrucciones necesitan una memoria interna que vaya traduciendo las microinstrucciones en señales eléctricas.

Hoy en día se usa una mezcla de las dos.

Criterios de valoración. Cada pregunta individual vale 0,5 puntos si está correcta (respuesta razonada y que conteste a lo que se pregunta). 0 si no está correcta o es incompleta.